

Cryptography

Summary note, Lectured by Zhandry

Hojune Lee

School of Computing, KAIST

Contents

1	Introduction to Cryptography	3
1.1	Encryption	3
1.2	Pseudo Random Functions	5
1.3	CPA-secure secret key encryption with PRF	6
2	Cryptography From Minimal Assumptions	8
2.1	Pseudo Random Generators	8
2.2	One Way Functions and One Way Permutations	13
3	Statistical Distance and Entropy	17

Chapter 1

Introduction to Cryptography

1.1 Encryption

This is study note of Prof. Zhandry's lecture[2]. There will be many notational things and remarks and pre-requested things. Since this is just 'summary note of summary note(or more depth)', many of things will be assumed. For here, I'm familiar with basic undergraduated mathematics and computation theory(e.g., Turing machine) so I'll omit such details. Let's begin.

Encryption(Symmetric key or Secret Key)

(Symmetric key or Secret key) Encryption scheme is pair (ENC, DEC) of two (probabilistic) polynomial time algorithms such that

$$\text{ENC} : (\text{key}, \text{plain text}) \mapsto \text{cipher text}$$

$$\text{DEC} : (\text{key}, \text{cipher text}) \mapsto \text{plain text}$$

In symmetric key encryption scheme, assume that both side users have common key k . Then what we want to guarantee is written by:

$$\Pr \left[\text{DEC}(k, \text{ENC}(k, m)) = m, \underbrace{k \xleftarrow{\$} \{0, 1\}^\lambda}_{\text{environment}} \right] = 1$$

Means that, under the setting that k is purely random among $\{0, 1\}^\lambda$ where λ is key length, above event happens almost always.

In Cryptography, we interested about security of such schemes. Since the word 'security' is quite abstracted, there are bunch of things to define. First, we must define our problem (or settings). In security problem, we always assume there exists unknown adversary A . Then the settings here can be:

- Could A obtain the full cipher text?

- What's the goal of A ? Obtain full plain text, or some function of the plain text?
- Could A know the ENC, DEC and exploit them?
- ...

Our goal is analyze under these various assumptions. Let's see the first example.

Indistinguishability Chosen Plaintext Attck

IND-CPA-EXP $_b(A, \lambda)$

Let A be an adversary, C be a challenger. b is parameter bit, here we assume that A never access to b .

1. C chooses purely(uniformly) random key $k \xleftarrow{\$} \{0, 1\}^\lambda$
2. A can choose any 2 plain text message m_0, m_1 then send both of them to C .
3. C select m_b and then construct $c \leftarrow \text{ENC}(k, m_b)$ then send it to A .
4. A can repeat above step arbitrary. Each time of repitition, we count A one unit of time.
5. Finally, A guess b' for b .

Intuitively, b is sort of information that C hiding. If A can guess correctly with high probability, the encryption scheme reveal few information about C 's secret to A . So, the ideal security under this setting can be defined by following.

Indistinguishable under Chosen Plaintext Attack

An encryption scheme (ENC, DEC) is IND-CPA secure(indistinguishable under a chosen plaintext attack) when for all adversary A who have polynomial resources limitation,

$$\left| \Pr[1 \leftarrow \text{IND-CPA-EXP}_0(A, \lambda)] - \Pr[1 \leftarrow \text{IND-CPA-EXP}_1(A, \lambda)] \right| < \varepsilon(\lambda)$$

Means that, two probability that A is wrong or correct is almost equal.

We often call such scheme by CPA secure. However, deterministic ENC can not be CPA secure. A can send (m_0, m_0) and (m_1, m_1) . Then A obtain c_0, c_1 which are deterministic w.r.t. m_0, m_1 . Then A can send (m_0, m_1) then can obtain correct b with probability 1.

1.2 Pseudo Random Functions

Since we've seen that deterministic is not suitable for security. So we need any sort of notion of Random. Here, we define what is computable random(pseudo random), and pure random.

Pseudo Random Function

Pseudo Random Function(PRF) is a deterministic polynomial time computable function such that

$$\text{PRF} : \{0, 1\}^\lambda \times \{0, 1\}^{n(\lambda)} \rightarrow \{0, 1\}^{m(\lambda)}$$

It can be understood by PRF maps $(\text{key}, \text{seed}) \mapsto \text{Random string}$. To it work as valid and secure random function, we do similar thing to previous section. Let's define following game.

Pseudo Random Function Experiments

$\text{PRF-EXP}_b(A, \lambda)$

Let A be an PPT adversary, C be a challenger. b is parameter bit, here we assume that A never access to b .

1. If $b = 0$, C choose purely random key $k \xleftarrow{\$} \{0, 1\}^\lambda$. If $b = 1$, C initializes an empty list L .
2. A choose any seed $x \in \{0, 1\}^{n(\lambda)}$ then send it to C . Then, C responds as follow:
 - If $b = 0$, C return $y \leftarrow \text{PRF}(k, x)$
 - If $b = 1$, if $(x, y) \in L$ then return y else generate pure random y and $L \leftarrow L :: (x, y)$
3. A can repeat this process arbitrary and one repition charge one time unit.
4. Finally, A guess b' for b .

In other words, the goal of A is distinguish computed random and pure random in polynomial time. If it is possible, then such PRF is not so good. So, we define secure PRF via above experiment.

PRF secure

A PRF is called PRF secure if for any PPT adversary A ,

$$\left| \Pr[1 \leftarrow \text{PRF-EXP}_0(A, \lambda)] - \Pr[1 \leftarrow \text{PRF-EXP}_1(A, \lambda)] \right| < \varepsilon(\lambda)$$

1.3 CPA-secure secret key encryption with PRF

Then we can construct encryption scheme with such PRF. Let's see the following construction.

Encryption with PRF

Let's assume that PRF is given.

- $\text{ENC}(k, m)$: Given key $k \in \{0, 1\}^\lambda$, plain text $m \in \{0, 1\}^{m(\lambda)}$. Choose $r \in \{0, 1\}^{n(\lambda)}$ randomly (pure or pseudo) then compute $c \leftarrow \text{PRF}(k, r) \oplus m$. Output of encryption is (r, c) .
- $\text{DEC}(k, (r, c))$: Given key $k \in \{0, 1\}^\lambda$, cipher text $c \in \{0, 1\}^{m(\lambda)}$ and chosen seed $r \in \{0, 1\}^{n(\lambda)}$. Compute $m' \leftarrow \text{PRF}(k, r) \oplus c = \text{PRF}(k, r) \oplus (\text{PRF}(k, r) \oplus m) = m$.

Then what we want to show is that above encryption is CPA-secure.

Encryption with PRF is CPA-secure

If given PRF is PRF-secure and $2^{n(\lambda)}$ is super-polynomial, then above (ENC, DEC) is CPA-secure.

Proof. Suppose not. Then since (ENC, DEC) is not CPA-secure, we can write

$$\left| \Pr[1 \leftarrow \text{IND-CPA-EXP}_0(A, \lambda)] - \Pr[1 \leftarrow \text{IND-CPA-EXP}_1(A, \lambda)] \right| \geq \varepsilon(\lambda)$$

Our goal is breaking the PRF-security of given pseudo random function with this inequality. For intuitive idea, our goal is construct following.

1. We'll define 2 more games, called H_1, H_2 . Let $\text{IND-CPA-EXP}_0 = H_1$ and $\text{IND-CPA-EXP}_1 = H_3$.

2. Then, we already have

$$\left| \Pr[1 \leftarrow H_0] - \Pr[1 \leftarrow H_3] \right| \geq \varepsilon(\lambda)$$

3. Then, we can obtain

$$\left| \underbrace{\Pr[1 \leftarrow H_0] - \Pr[1 \leftarrow H_1]}_A + \underbrace{\dots}_B + \underbrace{\Pr[1 \leftarrow H_2] - \Pr[1 \leftarrow H_3]}_C \right| \geq \varepsilon(\lambda)$$

4. Then, $|A| + |B| + |C| \geq |A + B + C| \geq \varepsilon(\lambda)$, implies that

$$\exists i \in \{0, 1, 2\}, \left| \Pr[1 \leftarrow H_i] - \Pr[1 \leftarrow H_{i+1}] \right| \geq \frac{1}{3}\varepsilon(\lambda)$$

5. Finally, all each cases contradicts to PRF-security.

Then what we need to do is that, define H_1, H_2 and show all cases $i \in \{0, 1, 2\}$ fails.

- H_0 : A sends m_0, m_1 to C . Then C choose m_0 , and choose random $r \in \{0, 1\}^{n(\lambda)}$, then send A back to $(r, m_0 \oplus \text{PRF}(k, r))$
- H_1 : C initialize $L = \emptyset$. A sends m_0, m_1 to C . C choose random $r \in \{0, 1\}^{n(\lambda)}$ then search (r, y) in L . If not exists, then generate pure random y and $L \leftarrow L \cup (r, y)$ and return $(r, m_0 \oplus y)$
- H_2 : C initialize $L = \emptyset$. A sends m_0, m_1 to C . C choose random $r \in \{0, 1\}^{n(\lambda)}$ then search (r, y) in L . If not exists, then generate pure random y and $L \leftarrow L \cup (r, y)$ and return $(r, m_1 \oplus y)$
- H_3 : A sends m_0, m_1 to C . Then C choose m_1 , and choose random $r \in \{0, 1\}^{n(\lambda)}$, then send A back to $(r, m_1 \oplus \text{PRF}(k, r))$

All of these are quite variance of IND-CPA-EXP game. Now, our remaining goal is that if one of H_i, H_{i+1} is distinguishable, then it contradicts to PRF-security.

1. ($i = 0, 2$) Here, suppose that A has power:

$$\left| \Pr[1 \leftarrow H_0] - \Pr[1 \leftarrow H_1] \right| \geq \frac{1}{3}\varepsilon(\lambda)$$

A can break the PRF-security of given PRF. Now B join the $\text{PRF-EXP}_b(A, \lambda)$. Then, A prepare (m_0, m_1) and send it to B . B generate random string $r \in \{0, 1\}^{n(\lambda)}$ and send B 's challenger. If $b = 0$, it gives pseudo random y , or $b = 1$, it gives pure random y . Anyway, B make $(r, y \oplus m_0)$ then give it back to A . Then A will distinguish y is pseudo random or pure random with probability $\varepsilon(\lambda)/3$. Then, B takes the output of A and submit it to its challenger. Then B also distinguish pseudo random and pure random with probability $\varepsilon(\lambda)/3$, contradiction. Vice versa for $i = 2$.

2. ($i = 1$) Here, suppose that A has power:

$$\left| \Pr[1 \leftarrow H_1] - \Pr[1 \leftarrow H_2] \right| \geq \frac{1}{3}\varepsilon(\lambda)$$

Here, note that A can distinguish $m_0 \oplus y$ and $m_1 \oplus y$ with probability $\varepsilon(\lambda)/3$. However, y is pure random string with $m(\lambda)$ length. Since y is purely random, $m_0 \oplus y, m_1 \oplus y$ both are also purely random strings. So it is generally impossible to distinguish them. One possible case is that, r is small so there can be collision in y . By the mechanism, we can possibly obtain $2^{n(\lambda)}$ amount of y so the collision probability is $2^{-n(\lambda)}$. Since $2^{n(\lambda)}$ is super polynomial, this probability is negligible. A can generate q queries, collision probability is $q^2/2^{n(\lambda)}$ which is negligible. So A can't distinguish H_1, H_2 , this case is impossible.

□

Chapter 2

Cryptography From Minimal Assumptions

In the previous chapter, we've seen that how can construct basic encryption via secure PRF. By the way, we just assume that PRF is just a mapping from $\{0, 1\}^\lambda \times \{0, 1\}^{n(\lambda)}$ into $\{0, 1\}^{m(\lambda)}$ which satisfy PRF security:

$$|\Pr[1 \leftarrow \text{PRF-EXP}_0(A, \lambda)] - \Pr[1 \leftarrow \text{PRF-EXP}_1(A, \lambda)]| < \varepsilon(\lambda)$$

Here $1 \leftarrow \text{PRF-EXP}_b(A, \lambda)$ means that A decide that itself gets some value via purely random manner. Natural wondering is that how PRF works.

2.1 Pseudo Random Generators

Pseudo Random Generator

PRG G is deterministic polynomial time computable function such that

$$G : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{\lambda+s(\lambda)}, \quad s(\lambda) \geq 1$$

Secure PRG

Let A be a PPT adversary who tries to determine whether given string in $\{0, 1\}^{\lambda+s(\lambda)}$ is randomly generated or deterministically generated.

$$A(q \in \{0, 1\}^{\lambda+s(\lambda)}) = \begin{cases} 0 & \text{A think } q \xleftarrow{\$} \{0, 1\}^{\lambda+s(\lambda)} \\ 1 & \text{A think } \exists G : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{\lambda+s(\lambda)}, q = G(x) \end{cases}$$

Then, we say that PRF G is secure when following inequality holds for negligible function ε :

$$\left| \Pr[A(G(x)) = 1 : x \xleftarrow{\$} \{0, 1\}^\lambda] - \Pr[A(y) = 1 : y \xleftarrow{\$} \{0, 1\}^{\lambda+s(\lambda)}] \right| < \varepsilon(\lambda)$$

This means that, for secure PRF G , any PPT adversary can't determine whether his query is randomly generated or deterministically generated by some G .

Remark?

If A has exponential power, than try $\mathcal{O}(2^\lambda)$ times and if A saw quite amount of collisions, then determine that query is deterministically generated. This probability is non-negligible.

As you can see from the codomain of PRG, one deterministic way to generate $s(\lambda)$ bits longer string from $\{0, 1\}^\lambda$ might be precise inductive extension. Let's define

$$G : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{\lambda+1}$$

works with deterministic mappings $G_0 : \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda, b : \{0, 1\}^\lambda \rightarrow \{0, 1\}$.

$$G : x \mapsto (G_0(x), b(x))$$

Then we can define inductively extension $G_s : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{\lambda+s}$ via:

$$G_s : x \mapsto (x_s, o_1, \dots, o_s)$$

where

$$x_0 = x, \quad x_i = G_0(x_{i-1}), \quad o_i = b(x_{i-1})$$

In other words, G_s is inductive chain of one-bit extension. We can prove that if such one-bit extension is secure PRG, then G_s also.

PRG Security Theorem

If one bit extension G is secure PRG, then G_s is also secure PRG for any $s \in \mathbb{N}$.

Proof. We'll use Hybrid Experiment Scheme here also. Let's denote the working of G_s in this:

$$\{0, 1\}^\lambda \xrightarrow{\$} x_0 \rightarrow (x_1 = G_0(x_0), o_1 = b(x_0)) \rightarrow \dots \rightarrow (x_s, o_1, \dots, o_s)$$

Then, we can imagine slight variance of this construction for $1 \leq i \leq s$.

$$\underbrace{\left(\{0, 1\}^\lambda \xrightarrow{\$} x_i, \{0, 1\} \xrightarrow{\$} o_1, \dots, o_i \right)}_{\text{Intercept Step}} \rightarrow (x_{i+1} = G_0(x_i), o_{i+1} = b(x_i)) \rightarrow \dots \rightarrow (x_s, o_1, \dots, o_s)$$

We call each of them are HYBRID_i . Here, $1 \leftarrow \text{HYBRID}_i(A, y)$ is A decide this string y is generated via HYBRID_i method. Note that HYBRID_s is uniformly random choice of entire $(x_s, o_1, \dots, o_s) \in \{0, 1\}^{\lambda+s}$. And denote original G_s be HYBRID_0 .

Now, suppose that G_s is non-secure PRG. Then HYBRID_0 and HYBRID_s is distinguishable and following holds:

$$\left| \Pr[A(G_s(x)) = 1 : x \xleftarrow{\$} \{0, 1\}^\lambda] - \Pr[A(y) = 1 : y \xleftarrow{\$} \{0, 1\}^{\lambda+s(\lambda)}] \right| \geq \varepsilon(\lambda)$$

However, we can denote this into what we've defined:

$$\begin{aligned}
& \left| \Pr[A(G_s(x)) = 1 : x \xleftarrow{\$} \{0, 1\}^\lambda] - \Pr[A(y) = 1 : y \xleftarrow{\$} \{0, 1\}^{\lambda+s(\lambda)}] \right| \geq \varepsilon(\lambda) \\
& \implies \left| \Pr[1 \leftarrow \text{HYBRID}_0(A, y)] - \Pr[1 \leftarrow \text{HYBRID}_s(A, y)] \right| \geq \varepsilon(\lambda) \\
& \implies \left| \sum_{i=0}^{s-1} \Pr[1 \leftarrow \text{HYBRID}_{i+1}(A, y)] - \Pr[1 \leftarrow \text{HYBRID}_i(A, y)] \right| \geq \varepsilon(\lambda) \\
& \implies \text{for some } i, |\Pr[1 \leftarrow \text{HYBRID}_{i+1}(A, y)] - \Pr[1 \leftarrow \text{HYBRID}_i(A, y)]| \geq \frac{\varepsilon(\lambda)}{s}
\end{aligned}$$

With this, now we'll show that this breaks the PRG security of G . Let B be another adversary for breaking G . Suppose that B gets $y \in \{0, 1\}^{\lambda+1}$. Goal of B is determining whether $y \xleftarrow{\$} \{0, 1\}^{\lambda+1}$ or $y = G(x), x \xleftarrow{\$} \{0, 1\}^\lambda$. In perspective of B , let

$$y = (x_{i+1}, o_{i+1}), \quad x_{i+1} \in \{0, 1\}^\lambda, o_{i+1} \in \{0, 1\}$$

Now B choose i bits for:

$$B \text{ choose : } o_1, \dots, o_i \xleftarrow{\$} \{0, 1\}$$

And since B knows and access to G , B can compute

$$x_{i+2} = G_0(x_{i+1}), o_{i+2} = b(x_{i+1}) \rightarrow \dots \rightarrow x_s = G_0(x_{s-1}), o_s = b(x_{s-1})$$

So now B obtain

$$z = (\underbrace{x_s}_{\text{computed}}, \underbrace{o_1, \dots, o_i}_{\text{uniform random}}, \underbrace{o_{i+1}}_{\text{query}}, \underbrace{o_{i+2}, \dots, o_s}_{\text{computed}})$$

B send this for A . Since A can distinguish HYBRID_i and HYBRID_{i+1} , when A see z ,

$$\begin{aligned}
\frac{\varepsilon(\lambda)}{s} & \leq |\Pr[1 \leftarrow \text{HYBRID}_{i+1}(A, z)] - \Pr[1 \leftarrow \text{HYBRID}_i(A, z)]| \\
& = \left| \Pr[A \text{ decide } z \text{ is generated by } \text{HYBRID}_{i+1}] \right. \\
& \quad \left. - \Pr[A \text{ decide } z \text{ is generated by } \text{HYBRID}_i] \right| \\
& = \left| \Pr[A \text{ decide } x_{i+1}, o_{i+1} \text{ is randomly chosen}] \right. \\
& \quad \left. - \Pr[A \text{ decide } x_{i+1}, o_{i+1} \text{ is computed via } G] \right|
\end{aligned}$$

Now B catch output of A , then B also decide $y = (x_{i+1}, o_{i+1})$ is randomly generated or computed via G with probability larger than $\varepsilon(\lambda)/s$. So,

$$\left| \Pr[B(G(x)) = 1 : x \xleftarrow{\$} \{0, 1\}^\lambda] - \Pr[B(y) = 1 : y \xleftarrow{\$} \{0, 1\}^{\lambda+1}] \right| \geq \frac{\varepsilon(\lambda)}{s}$$

So this is contradiction to G is secure PRF. So, G_s is also secure PRF. $\square$

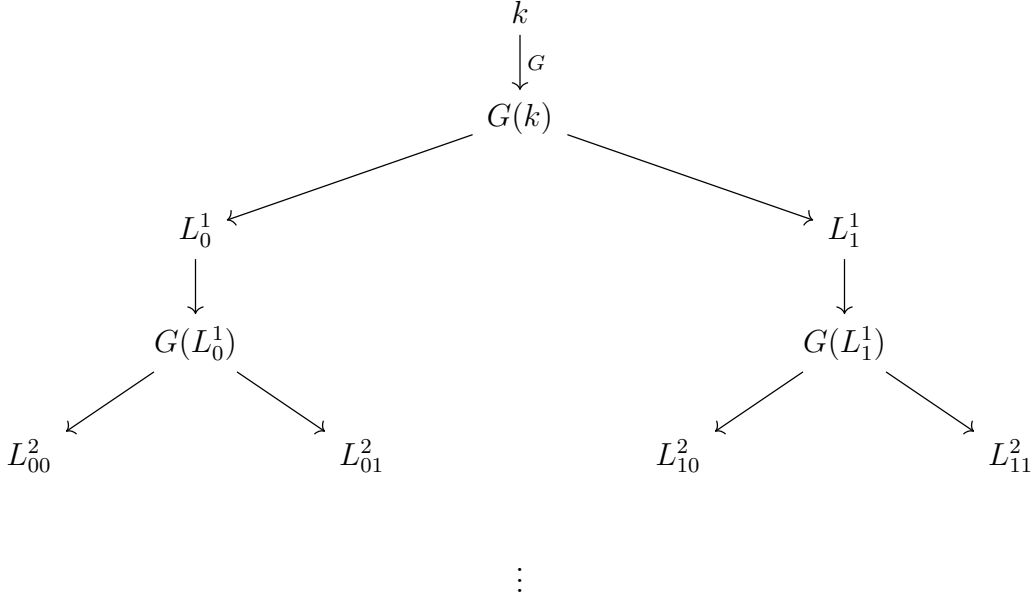
Then, now we'll see the example of construction of PRF from PRG. Choose one key:

$$k \in \{0, 1\}^\lambda$$

Then suppose that our Secure Pseudo Random Generator G be length double-expansion function:

$$G : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{2\lambda}$$

With this PRG, we can generate $2^{n(\lambda)} \cdot \lambda$ bits string:



So, in depth $n(\lambda)$, we can make $2^{n(\lambda)}$ of λ -length bits strings. With this, our PRF works:

$$\text{PRF} : \{0, 1\}^\lambda \times \{0, 1\}^{n(\lambda)} \rightarrow \{0, 1\}^\lambda$$

Explicitly,

$$\text{PRF} : (k, s) \mapsto L_s^{n(\lambda)} \in \{0, 1\}^\lambda$$

Although the entire space of co-domain is 2^λ , when k is fixed, the number of leaves is $2^{n(\lambda)}$. However we can compute efficiently when we know s . For each i -th branch, if $s_i = 0$ then choose left and $s_i = 1$ then choose right of given 2λ length output of G .

Secure PRG makes Secure PRF

When G is secure PRG, then above PRF is secure PRF.

Remark

We can't apply Hybrid Experiment method here directly since we need $2^{n(\lambda)}$ hybrid experiments, and then $\varepsilon' > \varepsilon/2^{n(\lambda)}$ also can be negligible.

Note that we can compute specific leaf in polynomial time as mentioned. Thus the time complexity to compute PRF value is $\mathcal{O}(n(\lambda) \cdot \text{time}(G))$ so the running time of PRF is polynomial. Now let's prove that if G is secure PRG, then above PRF is also secure.

Length Doubling PRF Construction is Secure
If G is secure PRG, then above PRF construction is secure.

Proof. Suppose PRF is not secure, then adversary A can obtain following advantage:

$$\left| \Pr[1 \leftarrow \text{PRF-EXP}_0(A, \lambda)] - \Pr[1 \leftarrow \text{PRF-EXP}_1(A, \lambda)] \right| \geq \varepsilon(\lambda)$$

Now we'll define hybrid game $H_0, \dots, H_n$ by following way.

$$H_i := \begin{cases} \text{Game with Original PRF} & i = 0 \\ \text{Replace } L^1, \dots, L^i \text{ into uniform random string} & 1 \leq i \leq n \end{cases}$$

In other words, we can view H_i as $n - i$ times application of PRG G mechanically. Then we can rewrite $\text{PRF-EXP}_0(A, \lambda)$ as H_0 and $\text{PRF-EXP}_1(A, \lambda)$ as H_n . So our assumption implies that A can distinct H_0 and H_n , so by triangular inequality, there exists $i \in [n]$ such that

$$\left| \Pr[1 \leftarrow H_{i-1}(A)] - \Pr[1 \leftarrow H_i(A)] \right| \geq \frac{\varepsilon(\lambda)}{n}$$

So A can distinct each 2^i of λ -length string is randomly generated or generated from $i - 1$ step. 2^{i-1} PRG is participated here so we can't use hybrid method directly. The key idea is that: A is polynomial adversary, so A can distinguish H_{i-1} and H_i with at most polynomial queries. Here exists 2 possible cases and we'll show how to prove for each case.

1. When A first commits total $q(\lambda)$ queries and then send it to challenger.
2. When A send total $q(\lambda)$ queries without pre-commits.

In the first case, we can know at most $q(\lambda)$ of L^{i-1} nodes that A will touch during challenging. Let's denote such strings by

$$L_1^{i-1}, \dots, L_m^{i-1}, \quad m \leq q(\lambda)$$

Construct m hybrid games $H_i^0, \dots, H_i^m$ by H_i^j : child of $L_1^{i-1}, \dots, L_j^{i-1}$ is generated by PRG G and remainings are purely random. Then, H_{i-1} is same with H_i^m and H_i is same with H_i^0 . So, A can distinguish H_i^j, H_i^{j-1} for some $j \in [m]$

$$\left| \Pr[1 \leftarrow H_i^j(A)] - \Pr[1 \leftarrow H_i^{j-1}(A)] \right| \geq \frac{\varepsilon(\lambda)}{n \cdot m} \geq \frac{\varepsilon(\lambda)}{n \cdot q(\lambda)}$$

So, A can distinct H_i^j and H_i^{j+1} efficiently. Then the PRG G adversary B can exploit A to break PRG security of G . B gets $Z \leftarrow \{0, 1\}^{2\lambda}$ and goal is distinguish Z is from pure random generation or G . Now, B construct PRF simulation by child of L_j^{i-1} be Z . Then A can distinguish Z non-negligibly, so obtaining

$$\left| \Pr[1 \leftarrow B(y), y \xleftarrow{\$} \{0, 1\}^{2\lambda}] - \Pr[1 \leftarrow B(G(x)), x \xleftarrow{\$} \{0, 1\}^\lambda] \right| \geq \frac{\varepsilon(\lambda)}{n \cdot q(\lambda)}$$

Second condition is called Adaptive Queries condition and need more tricks. We'll define $L_1^{i-1}, \dots, L_m^{i-1}$ where L_j^{i-1} means that j -th new arrival of level $i - 1$ during simulation of A . We don't need to exact spot that A distinct, but we can assume that A can distinct when k -th new arrival of $i - 1$ level. Under this, we can simulate B as similar with above and obtaining B breaks security of G , contradiction.

□

2.2 One Way Functions and One Way Permutations

We studied PRG to construct PRF. However, there are also another mystery: How can we implement PRG? In this section, we define one-way functions and permutations and then implement PRG.

One Way Function

A *one-way function* is a deterministic poly-time function $f : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{n(\lambda)}$ such that for any PPT adversary A exists negligible ε such that

$$\Pr[f(A(f(x))) = f(x), x \xleftarrow{\$} \{0, 1\}^\lambda] < \varepsilon(\lambda)$$

Means that, PPT adversary A tries to find pre-image of $f(x)$ such that randomly chosen but success probability is negligible. We call that f is *one-way permutation* if f is bijective and $n(\lambda) = \lambda$. In this case, A tries to find unique pre-image of $f(x)$.

Goal of This Section

Implement PRG via one-way permutation(OWP) and 'hardcore bit'.

Hardcore bit

Let $f : X \rightarrow Y$ be a one-way function. A hardcore bit for f is a function $hc_f : X \rightarrow \{0, 1\}$ such that for all PPT adversary A there exists ε negligible such that

$$\left| \Pr[1 \leftarrow A(f(x), hc_f(x)) : x \leftarrow X] - \Pr[1 \leftarrow A(f(x), b) : x \leftarrow X, b \leftarrow \{0, 1\}] \right| < \varepsilon(\lambda)$$

That means that, f is (intuitively) independent with such one bit. f does not be any hint to determine this bit is generated from purely random or hardcore bit function.

We already see the basic building blocks of PRG, which was G_0 and b . We'll explain this explicitly via OWP and hardcore bit. Let f be one-way permutation and hc is its hardcore bit function. Then $G(x) = (f(x), hc(x))$ becomes one-bit stretch PRG. This PRG is secure. Since f is one-way permutation, if x is pure randomly chosen, we can assume that $f(x)$ is also purely random bits string. So that, A can't distinct $f(x)$ and

purely random λ bits string, and last hardcore bit also.

Remarks

Are there any hc function such that hardcore for any OWP? Answer is no. Let f be any OWF. Then we construct new OWF $f'(x) = f(x) || hc(x)$. Then adversary easily know that the last bit of $f'(x)$ is hardcore bit. So, hc is not hardcore bit for f' .

Now we state the main important theorem, to claim that every OWF has proper hardcore bit function. (So that for any OWF f , we can find hc_f and construct PRG by $G(x) = (f(x), hc_f(x))$.)

Goldreich-Levin Theorem

Let $F : \{0, 1\}^n \rightarrow Y$ be a OWF. Define $F' : \{0, 1\}^{2n} \rightarrow \{0, 1\}^n \times Y$ such that $F'(x, r) = (r, F(x))$. Then, $\langle x, r \rangle = \sum x_i r_i \pmod{2}$ is a hardcore for F' .

Proof. Suppose not. Then, some PPT adversary A can break hardcoreness of $\langle x, r \rangle$ and obtain following inequality:

$$\left| \Pr[A(r, F(x), \langle x, r \rangle) = 1] - \Pr[A(r, F(x), b) = 1] \right| > \varepsilon(n)$$

Where $r \xleftarrow{\$} \{0, 1\}^n$, $x \xleftarrow{\$} \{0, 1\}^n$ and $b \xleftarrow{\$} \{0, 1\}$. Without loss of generality,

$$\Pr[A(r, F(x), \langle x, r \rangle) = 1] - \Pr[A(r, F(x), b) = 1] > \varepsilon(n)$$

Now let's construct new agent B . B takes $r, F(x) \in \{0, 1\}^n$ and pure-randomly choose $t \in \{0, 1\}$. Then run $A(r, F(x), t)$.

$$B(r, F(x)) = \begin{cases} t & A(r, F(x), t) = 1 \\ 1 - t & A(r, F(x), t) = 0 \end{cases}$$

Then B guess the hardcore function of given instance $r, F(x)$. Since A has $\varepsilon(n)$ advantage to determine whether given instance is from hardcore function or pure random,

$$\Pr[B(r, F(x)) = \langle x, r \rangle] > \frac{1}{2} + \varepsilon(n)$$

This is because that, if $t = \langle x, r \rangle$ then $A(r, F(x), t) = 1$ has higher probability. If $1 - t = \langle x, r \rangle$, then $A(r, F(x), t) = 0$ has higher probability. So, B can guess $\langle x, r \rangle$ by exploiting A with $\varepsilon(n)$ advantage. With agent B , our goal is breaking the OWF property of F . If $\varepsilon(n) = \frac{1}{2}$, it implies that B always answer correctly. Then by standard basis bit string e_i , we take n queries of $B(e_i, x) = \langle x, e_i \rangle = x_i$ and obtain x . If $\varepsilon(n) = \frac{1}{2} - \frac{1}{2n}$, then choose n random bit string $r_1, \dots, r_n$ and compute $B(r_i, F(x))$. They correctly answer $\langle x, r_i \rangle$ with probability $1 - \frac{1}{2n}$, so total-correct probability is $\frac{1}{2}$. If they are all correct, we

can observe n linear system of $\langle x, r_i \rangle = b_i$. Solving this gives us total x with probability $1/2$. Third, suppose that $\varepsilon(n) = \frac{1}{4} + \gamma$ where γ is non-negligible. Then, we can denote

$$\Pr[B(r, F(x)) = \langle x, r \rangle : x, r \xleftarrow{\$} \{0, 1\}^n] = \frac{3}{4} + \gamma$$

Then we can view this as joint distribution of x, r , so let's define

$$p(x) = \Pr[B(r, F(x)) = \langle x, r \rangle : r \xleftarrow{\$} \{0, 1\}^n]$$

By definition,

$$\mathbb{E}_{x \xleftarrow{\$} \{0, 1\}^n} [p(x)] = \frac{3}{4} + \gamma$$

Let's denote $Y(x) = 1 - p(x)$ then we can rewrite above into

$$\mathbb{E}_{x \xleftarrow{\$} \{0, 1\}^n} [Y(x)] = \frac{1}{4} - \gamma$$

$Y(x)$ denotes the probability that B guess wrong answer for fixed x and uniform r . So, if there exists $x \in \{0, 1\}^n$ such that $Y(x) < \frac{1}{4} - \frac{\gamma}{2}$, we call such x is 'good'. If $Y(x) \geq \frac{1}{4} - \frac{\gamma}{2}$ then such x is called 'bad'.

Lemma 2.2.1. (*Markov's Inequality*) For random variable $X : \Omega \rightarrow \mathbb{R}$ and function $f : \mathbb{R} \rightarrow \mathbb{R}^+$,

$$P(f(X) \geq a) \leq \frac{\mathbb{E}[f(X)]}{a}$$

Then,

$$\Pr[x \text{ is bad}] = \Pr[Y(x) \geq \frac{1}{4} - \frac{\gamma}{2}] \leq \frac{\mathbb{E}[Y(x)]}{\frac{1}{4} - \frac{\gamma}{2}} = \frac{1 - 4\gamma}{1 - 2\gamma}$$

So,

$$\Pr[x \text{ is good}] \geq 1 - \frac{1 - 4\gamma}{1 - 2\gamma} = \frac{2\gamma}{1 - 2\gamma} \geq 2\gamma$$

Means that we can choose good x via sampling with non-negligible probability. Now suppose that given x is good. Then the goal of adversary C is breaking OWF assumption of F . If the probability that given x is good is non-negligible and if x is good then C can compute x from $F(x)$ with non-negligible probability, then we can say that C has non-negligible advantage to break OWF assumption of F . Now B break OWF of F with good x . Let's define

$$H(r) = B(r, F(x))$$

Choose enough many $r \xleftarrow{\$} \{0, 1\}^n$ and C computes

$$H(r) \oplus H(r \oplus e_i)$$

Then, if B computes both of them correctly,

$$H(r) \oplus H(r \oplus e_i) = \langle x, r \rangle \oplus \langle x, r \oplus e_i \rangle = \langle x, e_i \rangle = x_i$$

So, C can obtain x_i correctly when B guess both of them correctly. So,

$$\begin{aligned}
 \Pr[C \text{ correctly guess } x_i \mid x \text{ is good}] &= \Pr[H(r) \oplus H(r \oplus e_i) = x_i] \\
 &= \Pr[B \text{ guess both correctly}] \\
 &= 1 - \Pr[B \text{ wrong one of them}] \\
 &\geq 1 - 2 \left(\frac{1}{4} - \frac{\gamma}{2} \right) \\
 &= \frac{1}{2} + \gamma
 \end{aligned}$$

So,

$$\begin{aligned}
 \Pr[F(C(F(x))) = F(x) \mid x \xleftarrow{\$} \{0,1\}^n] &\geq \Pr[C \text{ correct} \mid x \text{ is good}] \Pr[x \text{ is good}] \\
 &\geq \frac{1}{2} + \mathcal{O}(\gamma)
 \end{aligned}$$

C has non-negligible advantage to break OWF assumption of F , which is contradiction.

□

Chapter 3

Statistical Distance and Entropy

In this chapter, I reference Alex Lombardi's note[1]

Bibliography

- [1] Alex Lombardi. Lecture notes on cryptography. https://drive.google.com/file/d/1I5a0bjjX_OZ1pn97mNiALAsqbqVJDV7d/view, 2023.
- [2] Mark Zhandry. Lecture notes on cos533. <https://mzhandry.github.io/courses/2021-Spring-COS533/>, 2021.